

Take-home Assessment for Research Assistant Candidates

The project “Digital Governance in an Age of AI and Big Data” requires the Research Assistant to support empirical research, data processing, computational analysis, and research-tool development.

This take-home assessment evaluates whether the candidate can translate research needs into a working analytical tool. The assessment focuses on accuracy, usability, reproducibility, and research judgment.

Candidates may choose one of the following two tasks. However, **Task 2 is strongly preferred**, especially for candidates who wish to demonstrate stronger capacity in computational social science, text analysis, and configurational methods.

Task 1 is expected to be completed within **3 days**.

Task 2 is more advanced and is expected to be completed within **5 days**.

Candidates may use any appropriate programming language, package, or framework. The final product should be usable by a researcher through a simple interface. It should not require the reviewer to manually edit code to run the main workflow.

For submission, candidates are required to upload the full project to **GitHub as an open-source repository** and provide the repository link. The repository should include the source code, demo data, sample outputs, and clear instructions for installation and use.

Candidates are also strongly encouraged to provide a **web-accessible demo** of the tool. This may be hosted through **GitHub Pages, Streamlit Community Cloud, Hugging Face Spaces, Render, Vercel, Netlify**, or any other suitable platform. The reviewer should be able to access and test the main workflow through a simple web interface whenever possible.

A submission with a working Web UI and a public demo link will be viewed more favorably than a submission that only provides local code.

Task 1: Regression Analysis and Reporting Tool

Task objective

Build a research tool that allows a user to upload a tabular dataset, select variables, run regression models, and export clear tables and figures.

The tool should support a common applied social science workflow: from raw dataset to regression results, visual outputs, and exportable research materials.

User scenario

A researcher uploads a survey or administrative dataset. The researcher selects one dependent variable and several independent variables. The tool checks the data, suggests or allows suitable models, runs the analysis, and produces clean outputs that can be used in a report or academic paper.

Minimum expected functions

The tool should allow the user to:

Upload a dataset in a common tabular format.

Preview the dataset.

Select dependent and independent variables.

Run at least two common regression models, such as OLS and logit.

View regression results in a clean table.

Export the regression table in a format suitable for reporting.

Generate at least one meaningful figure.

View basic warnings about data problems.

Expected outputs

The final submission should include:

A working tool.

A demo dataset.

A short instruction file.

At least one sample regression table.

At least one sample figure.

A short explanation of the model choice and output interpretation.

Acceptance standards

A submission will be considered acceptable only if it meets the following standards.

First, the tool must run successfully with the provided demo dataset. The reviewer should be able to reproduce the sample outputs.

Second, the regression results must be statistically correct. Coefficients, standard errors, sample size, and model statistics should match standard statistical software outputs.

Third, the tool must make reasonable use of variable types. For example, it should not treat a clearly categorical variable as a continuous variable without warning.

Fourth, the tool must handle missing data transparently. It should show how many observations are dropped or used in each model.

Fifth, the regression table must be readable. It should include variable names, coefficients, standard errors, significance information, sample size, and model labels.

Sixth, the figure must add analytical value. It should not be a decorative chart. Suitable figures include coefficient plots, predicted-value plots, marginal-effect plots, or model-comparison plots.

Seventh, the interface must be usable by a non-programmer. The reviewer should be able to complete the main workflow without editing the code.

Eighth, the submission must include clear instructions. The reviewer should know how to install, open, and test the tool.

Ninth, the tool should include basic error handling. If the user selects unsuitable variables or uploads problematic data, the tool should give a clear message rather than crash.

Tenth, the candidate should briefly explain the assumptions and limitations of the tool.

Higher-level evaluation standards

A strong submission will show more than technical completion. It will show research judgment.

A strong submission may include automatic model suggestions. For example, it may recommend OLS for a continuous outcome and logit for a binary outcome. It may also explain why a model is not suitable.

A strong submission may include better reporting options, such as multiple model comparison tables, marginal effects, robust standard errors, or clearer export formats.

A strong submission may also show attention to design. The table and figures should look polished enough for academic use.

Scoring guide

Total: 100 points.

Accuracy of regression results: 25 points.

Usability of the interface: 15 points.

Quality of tables: 15 points.

Quality of figures: 10 points.

Data checks and warnings: 10 points.

Reproducibility: 10 points.

Code organization: 10 points.

Research judgment: 5 points.

Task 2: Text Classification to QCA Analysis Tool

Task objective

Build a research tool that converts raw text into QCA-ready conditions and produces basic QCA results.

The tool should allow a researcher to upload raw text data and conceptual prototypes. It should classify or score the texts against those prototypes, convert the scores into set-membership values, and generate QCA outputs.

User scenario

A researcher has citizen messages, government replies, policy texts, or public consultation comments. The researcher also has several conceptual categories, such as dissatisfaction, policy demand, co-production request, trust, or government responsiveness.

The tool should help the researcher turn raw text into conditions for configurational analysis.

Example input

The user may upload a text dataset like this:

case_id,text,outcome

1,我已经多次反映小区垃圾分类设施不足，希望政府尽快处理。,1

2,这个政策解释不清楚，居民不知道应该如何申请。,0

3,我们愿意配合街道办开展社区宣传活动。,1

The user may also provide prototype descriptions like this:

condition_name,prototype,type

dissatisfaction,The citizen expresses dissatisfaction anger or frustration,condition

policy_demand,The citizen asks for policy clarification adjustment or

implementation,condition

coproduction_request,The citizen shows willingness to cooperate with government,condition
responsiveness,The government provides clear concrete and respectful response,outcome

Minimum expected functions

The tool should allow the user to:

Upload a raw text dataset.

Select the text column.

Input or upload conceptual prototypes.

Score each text against each prototype.

Generate condition scores.

Convert the condition scores into fuzzy-set or crisp-set membership values.

Produce a QCA-ready dataset.

Generate a truth table.

Report consistency and coverage.

Produce at least one configuration result.

Generate at least one useful table or figure.

Export the main results.

Expected outputs

The final submission should include:

A working tool.

A demo text dataset.

A prototype file or example prototype input.

A similarity or classification score table.

A calibrated membership table.

A QCA-ready dataset.

A truth table.

Consistency and coverage results.

At least one solution configuration.

At least one sample figure.

A short explanation of calibration choices.

Acceptance standards

A submission will be considered acceptable only if it meets the following standards.

First, the tool must run successfully with the provided demo data. The reviewer should be able to reproduce the submitted outputs.

Second, the tool must clearly show how raw text is converted into condition scores. The process cannot be hidden or unexplained.

Third, the prototype-based scoring must be meaningful. Texts that are conceptually closer to a prototype should receive higher scores than clearly unrelated texts.

Fourth, the tool must produce a transparent calibration step. It should explain how raw scores become fuzzy-set membership values or crisp-set values.

Fifth, the calibration logic must be defensible. The candidate may use thresholds, percentiles, anchors, or other reasonable rules. However, the rule must be stated clearly.

Sixth, the tool must allow the reviewer to inspect intermediate results. The reviewer should be able to see raw scores, calibrated scores, and final QCA conditions.

Seventh, the QCA-ready dataset must be correctly structured. Each row should represent a case. Each condition should be a separate column. The outcome should be clearly identified.

Eighth, the truth table must be readable. It should show configurations, number of cases, outcome values, consistency, and coverage where applicable.

Ninth, the QCA metrics must be correctly calculated. Consistency and coverage should follow standard QCA logic.

Tenth, the tool must report at least one solution configuration. It should also identify contradictory or weak configurations if they appear.

Eleventh, the figure must help interpretation. Suitable figures include a membership heatmap, score distribution plot, truth-table plot, or consistency-coverage plot.

Twelfth, the interface must be usable by a researcher. The reviewer should not need to manually edit code to complete the main workflow.

Thirteenth, the submission must explain limitations. For example, prototype quality, text ambiguity, calibration choices, and small sample size may affect the results.

Higher-level evaluation standards

A strong submission will show awareness that this is not only a classification task. It is also a research-design task.

A strong submission will explain why a certain prototype produces a certain condition score.

A strong submission will allow users to adjust calibration thresholds.

A strong submission will support both fuzzy-set and crisp-set logic.

A strong submission will handle Chinese text well.

A strong submission will produce outputs that are clear enough for a research report.

Scoring guide

Total: 100 points.

Quality of text-to-condition scoring: 20 points.

Quality of calibration: 20 points.

Correctness of QCA outputs: 20 points.

Transparency of intermediate results: 10 points.

Usability of the interface: 10 points.

Quality of tables and figures: 10 points.

Reproducibility: 5 points.

Research judgment and limitation discussion: 5 points.

General Submission Requirements

Candidates should submit:

A project folder or repository.

A short instruction file.

A demo dataset.

Sample output files.

A brief technical note of around 500 to 1,000 words.

The technical note should answer:

What does the tool do?

What data does it require?

What outputs does it produce?

How should the results be interpreted?

What assumptions does the tool make?

What are the main limitations?

What would the candidate improve with more time?

General Evaluation Principles

The assessment will not reward complexity for its own sake. A simple tool that runs correctly, produces accurate results, and explains its assumptions clearly is better than an ambitious tool that is unstable or opaque.

The most important standards are:

The tool works.

The results are correct.

The workflow is understandable.

The outputs are readable.

The assumptions are clear.

The tool is useful for real research.